

DOCUMENTACIÓN TÉCNICA

Veterinaria+

Sistema de Gestión de Clínica Veterinaria

Documento: Manual de Usuario y Manual Técnico

Materia: Ingeniería de Software II

Profesor: Ing. Julio César Sarandigua Quintero

Repositorio: github.com/johan-bm/veterinaria-plus

Parte 1 — Manual de Usuario

Esta sección describe cómo navegar y utilizar los módulos principales de Veterinaria+ desde el navegador, una vez que la aplicación está en ejecución en `http://localhost:8080`.

1.1 Módulos disponibles

Módulo	Ruta	Funciones disponibles
Propietarios	/propietarios	Listar, registrar, editar y eliminar propietarios de mascotas.
Mascotas	/mascotas	Listar, registrar, editar y eliminar mascotas; búsqueda por nombre, especie o propietario.
Veterinarios	/veterinarios	Listar veterinarios activos, registrar nuevos y dar de baja (baja lógica).
Citas	/citas	Agendar, confirmar, cancelar y completar citas; validación automática de disponibilidad de horario.
Medicamentos	/medicamentos	Catálogo de medicamentos, control de stock e incremento de existencias.
Reportes	/reportes	Reporte de citas por período con estadísticas por estado.

1.2 Flujo típico de uso

1. Registrar al propietario en el módulo **Propietarios**.
2. Registrar la mascota en el módulo **Mascotas**, asociándola al propietario recién creado.
3. Agendar una cita en el módulo **Citas**, seleccionando la mascota y el veterinario disponible.
4. El día de la consulta, el veterinario confirma la cita y registra el historial clínico (síntomas, diagnóstico, notas).
5. Si la consulta requiere medicamentos, se prescriben desde el historial clínico; el stock se descuenta automáticamente.
6. Al guardar el historial, la cita pasa al estado *Completada* automáticamente.

1.3 Cambiar el idioma de la interfaz

La aplicación soporta español, inglés y francés. El idioma se cambia agregando el parámetro `?lang=es`, `?lang=en` o `?lang=fr` a cualquier URL, o usando el selector de idioma disponible en la barra de navegación. La preferencia se guarda en una cookie durante 30 días.

1.4 Validaciones y mensajes de error

Todos los formularios validan los datos antes de guardar. Si un campo es inválido (por ejemplo, un correo electrónico mal formado o un campo obligatorio vacío), el formulario se vuelve a mostrar con un mensaje de error específico junto al campo, traducido al idioma activo.

Nota: Si se intenta agendar una cita en un horario que ya está ocupado por el mismo veterinario, el sistema rechaza la operación y solicita elegir otro horario.

Parte 2 — Manual Técnico

Esta sección describe cómo instalar, ejecutar y verificar el proyecto desde el código fuente.

2.1 Requisitos previos

- Java 21 (JDK)
- Maven 3.9 (incluido en NetBeans o instalado de forma independiente)
- Docker Desktop (para levantar la base de datos PostgreSQL y/o la aplicación completa)
- Git

2.2 Instalación y ejecución — opción recomendada (Docker completo)

Esta opción levanta la aplicación y la base de datos PostgreSQL juntas, sin instalar nada más que Docker.

Comando	Descripción
<code>git clone https://github.com/johan-bm/veterinaria-plus.git</code>	Clona el repositorio.
<code>cd veterinaria-plus</code>	Entra a la carpeta del proyecto.
<code>docker compose up -d --build</code>	Construye y levanta los contenedores de la aplicación y la base de datos.

Una vez levantado, la aplicación queda disponible en `http://localhost:8080`.

2.3 Instalación — desarrollo local (NetBeans)

Esta opción es útil para desarrollar con recarga rápida, dejando solo la base de datos en Docker.

Comando	Descripción
<code>docker run -d --name veterinaria-db -p 5432:5432 -e POSTGRES_DB=veterinaria_plus -e POSTGRES_USER=veterinaria_user -e POSTGRES_PASSWORD=veterinaria_pass postgres:15</code>	Levanta solo la base de datos PostgreSQL en un contenedor.
<code>mvn spring-boot:run -Dspring-boot.run.profiles=local</code>	Ejecuta la aplicación localmente contra la base de datos en Docker.

2.4 Ejecutar las pruebas unitarias

Comando	Descripción
<code>mvn test</code>	Ejecuta la suite completa de pruebas unitarias e integración (JUnit 5 + Mockito).
<code>mvn clean test jacoco:report</code>	Ejecuta las pruebas y genera el reporte de cobertura HTML en <code>target/site/jacoco/</code> .
<code>mvn verify</code>	Ejecuta el ciclo completo: compilación, pruebas y verificación del umbral mínimo de cobertura.

2.5 Estructura general del código

Paquete	Contenido
<code>modelo</code>	Entidades JPA (Mascota, Propietario, Veterinario, Cita, HistorialClinico, Medicamento, Tratamiento, Especie, Especialidad).
<code>repositorio</code>	Interfaces Spring Data JPA con las consultas de acceso a datos.
<code>servicio / servicio.impl</code>	Interfaces y lógica de negocio de la aplicación.
<code>controlador</code>	Controladores Spring MVC que exponen las rutas web.
<code>dto</code>	Objetos de transferencia de datos entre la vista y los servicios.
<code>excepcion</code>	Excepciones de negocio personalizadas.
<code>config</code>	Configuración de auditoría JPA e internacionalización.

2.6 Verificación de estado (health check)

El proyecto expone Spring Boot Actuator en `/actuator/health`, utilizado por Docker Compose para verificar que la aplicación está respondiendo correctamente antes de marcarla como disponible.